

Data Validation Checklist

A repeatable pre-dashboard and pre-report quality check

Practical Tools & Guides | Built for real-world reference, learning, and repeatable work.

Core idea

Validation asks: 'How many records violate the rule?' For many checks, zero is the success condition.

1. Import completeness

- Row count matches the source expectation.
- Skipped rows = 0 unless intentional.
- Warnings = 0 or every warning has been investigated.
- Primary keys are unique.
- Expected nullable fields are NULL, not empty strings or zero dates.

2. Duplicate checks

```
SELECT business_key, COUNT(*) AS row_count
FROM table_name
GROUP BY business_key
HAVING COUNT(*) > 1;
```

For historical/event tables, test the correct grain, such as employee + date + event type.

3. Orphan checks

```
SELECT COUNT(*) AS orphan_rows
FROM child c
LEFT JOIN parent p
  ON c.parent_id = p.parent_id
WHERE p.parent_id IS NULL;
```

4. Cross-table consistency

```
SELECT COUNT(*) AS mismatches
FROM time_entries t
JOIN projects p
  ON t.project_id = p.project_id
WHERE t.customer_id <> p.customer_id;
```

5. Date logic

```
SELECT COUNT(*) AS invalid_dates
FROM projects
WHERE target_end_date < start_date
    OR actual_end_date < start_date;
```

6. Status logic

```
SELECT COUNT(*) AS invalid_status_rows
FROM support_tickets
WHERE status = 'Closed'
    AND resolved_date IS NULL;
```

Always validate against the exact status labels used by the business. A failed query can mean the rule is wrong, not the data.

7. Math reconciliation

```
SELECT COUNT(*) AS mismatches
FROM invoices
WHERE ROUND(invoice_total - amount_paid, 2)
    <> ROUND(balance_due, 2);

SELECT COUNT(*) AS rollup_mismatches
FROM invoices i
JOIN (
    SELECT invoice_id,
        ROUND(SUM(line_total),2) AS line_sum
    FROM invoice_lines
    GROUP BY invoice_id
) l ON i.invoice_id = l.invoice_id
WHERE ROUND(i.subtotal,2) <> l.line_sum;
```

8. Allowed ranges

```
SELECT COUNT(*) AS invalid_rows
FROM product_usage_monthly
WHERE active_user_pct < 0
    OR active_user_pct > 100;
```

9. NULL logic

- Closed records should have completion/resolution data where required.
- Open records should not have fields populated that imply completion.
- Optional fields should use NULL rather than invented placeholders.

- Do not confuse an empty string with NULL.

10. Reference-table checks

```
SELECT COUNT(*) AS unmatched
FROM expenses e
LEFT JOIN chart_of_accounts a
  ON e.account_number = a.account_number
WHERE a.account_number IS NULL;
```

11. Before a dashboard is trusted

- Source row counts reconcile.
- Relationships reconcile.
- Business-rule checks return expected results.
- Financial totals reconcile at detail and summary levels.
- Known data-quality exceptions are flagged, not silently corrected.
- Transformations are documented and repeatable.
- The final report query is tested after the model passes validation.

12. Quick validation template

```
-- 1. Count
SELECT COUNT(*) FROM table_name;

-- 2. Duplicates
SELECT key_column, COUNT(*)
FROM table_name
GROUP BY key_column
HAVING COUNT(*) > 1;

-- 3. Missing required relationships
SELECT COUNT(*)
FROM child c
LEFT JOIN parent p ON c.parent_id = p.parent_id
WHERE p.parent_id IS NULL;

-- 4. Invalid business rule
SELECT COUNT(*)
FROM table_name
WHERE <bad condition>;

-- 5. Reconcile totals
SELECT ...
```